



GenAI Basics

A Beginner's Course on Generative AI

Tokens · Prompts · Responses · APIs · Your First AI Script

Open-Source Learning Notes

What is Generative AI?

Generative AI (GenAI) refers to AI systems that create new content — text, code, images, audio, or video — instead of just analyzing existing data. When you ask a chatbot a question and it writes an answer in real time, that answer didn't exist anywhere before you asked; the model generated it word by word.

The most common form of GenAI today is text generation, powered by **Large Language Models (LLMs)**.

What is an LLM?

Large Language Model (LLM)

A neural network trained on massive amounts of text (books, websites, code) so it learns patterns of language. Given some input text, it predicts the most likely next piece of text, one small chunk at a time, and repeats this until it forms a full answer.

Example: Llama 3.3, GPT-4, Claude, and Gemini are all LLMs.

In this course, the examples use **Groq** — a platform that lets developers send text to an LLM (like Llama 3.3) over the internet and get a generated answer back, very quickly.

Why this matters for you

- Every chatbot, coding assistant, and AI writing tool is built on this same core loop: send text in → model predicts text out.
- Once you understand this loop, you can build your own AI-powered apps — chatbots, summarizers, financial assistants, and more.
- The rest of this course breaks that loop into its exact building blocks: prompts, tokens, responses, and API calls.

Prompts and Tokens

What is a Prompt?

Prompt

The text you send to the model — your question, instruction, or request. The model's entire job is to generate a good continuation of your prompt.

Example: "What is HEAP in DSA?" is a prompt.

A well-written prompt is specific and gives the model context. Vague prompts get vague answers; clear prompts get useful answers.

What is a Token?

Token

A token is a small chunk of text — often a word, part of a word, or a punctuation mark — that the model reads and generates one piece at a time. Models don't see full sentences at once; they see a sequence of tokens.

Example: "HEAP" might be one token. "Understanding" might split into two tokens: "Understand" + "ing".

Tokens matter for two practical reasons:

- **Cost:** Most AI APIs (including Groq) charge based on the number of tokens you send in and receive back — not per request.
- **Limits:** Every model has a maximum **context window** — the total number of tokens (prompt + response + conversation history) it can handle in one go. If you go over it, older text gets cut off or the request fails.

Rough rule of thumb: 1 token ≈ 4 characters of English text, or about ¾ of a word.

Responses, Roles, and Context

What is a Response?

Response

The object the API sends back after processing your prompt. It contains the generated answer plus metadata — which model was used, how many tokens were consumed, why the model stopped generating, and a unique request ID.

Example: `response.choices[0].message.content` holds the actual text answer.

What is a Role?

Every message sent to a chat-based LLM has a role that tells the model who is 'speaking':

- **system** — Instructions that set the model's behaviour or personality (often invisible to the end user).
- **user** — The person's message or question.
- **assistant** — The model's own previous replies, included so it remembers the conversation so far.

What is a Context Window?

Context Window

The 'memory span' of a single API call, measured in tokens. It includes the system message, every past user/assistant message you send, and the new response — all added together.

Example: A model with a 128k-token context window can hold roughly 90,000-100,000 words of conversation before it starts forgetting the earliest parts.

APIs, API Keys, and Environment Variables

What is an API?

API (Application Programming Interface)

A defined way for two programs to talk to each other. When you use an LLM 'API', your Python script sends a request over the internet to the provider's servers (like Groq), and the server sends back a response — all without you needing to know how the model works internally.

What is an API Key?

API Key

A secret password-like string that identifies you to the provider and authorizes your requests (and tracks your usage/billing). Anyone with your key can use your account, so it must never be shared or uploaded to public code (e.g. GitHub).

Why keep keys in a .env file?

Hardcoding a key directly into a script is risky — if you push that file to GitHub, your key becomes public within seconds. Instead, developers store secrets in a separate **.env** file that is excluded from version control (via `.gitignore`), and load it at runtime.

```
# .env file (never uploaded to GitHub)
API=gsk_your_secret_key_here
```

The **python-dotenv** library's `load_dotenv()` function reads this file and makes its variables available through `os.getenv()`, keeping secrets out of your source code entirely.

Code Walkthrough — Setup & Authentication

This chapter walks through the script below, piece by piece, connecting each line back to the concepts from Chapters 1–4.

Step 1 — Load the secret key

```
from dotenv import load_dotenv
import os
from groq import Groq

load_dotenv() # reads the .env file
my_api = os.getenv("API") # fetches your key

if not my_api:
    raise ValueError("No API found")
else:
    print("API Found!!!")
```

This is the authentication step described in Chapter 4 — it loads your API key safely and stops the script early with a clear error if the key is missing.

Step 2 — Create the client

```
client = Groq(api_key=my_api)
```

This creates a connection object. Every request you send afterward goes through this 'client', which automatically attaches your API key.

Step 3 — Choose a model

```
model_ = "llama-3.3-70b-versatile"
```

This tells Groq's servers which LLM (Chapter 1) should generate the response. Different models trade off speed, cost, and quality.

Code Walkthrough — Building the Prompt

Step 4 — Define the role and prompt

```
rolez = "user"  
prompt = "What is HEAP in DSA?"
```

rolez uses the 'user' role from Chapter 3 — this message represents you asking a question. *prompt* is the actual text (Chapter 2) sent to the model.

Step 5 — Package it into a messages list

```
messages = [  
  {  
    "role": rolez,  
    "content": prompt  
  }  
]
```

Groq's Chat Completion API always expects a *LIST* of role/content dictionaries — this is how it reconstructs the full conversation, including any past assistant replies, every time you make a call.

Why a list, not a single string?

Because conversations have turns. If you later wanted the model to 'remember' earlier messages, you'd simply append more role/content dictionaries to this same list — one for each past user message and each past assistant reply — before sending the next request.

Code Walkthrough — Sending & Reading the Response

Step 6 — Send the request

```
response = client.chat.completions.create(  
    model=model_,  
    messages=messages  
)
```

This is the actual API call (Chapter 4) — the script pauses here while the request travels to Groq's servers, the model generates tokens, and a full response object (Chapter 3) travels back.

Step 7 — Extract just the answer

```
print(response.choices[0].message.content)
```

The response object contains far more than the answer text — token counts, model name, a stop reason, and a request ID. This line drills down to just the generated text most applications actually show the user.

What response actually looks like

If you printed the raw response instead of just the text, you'd see something like this (simplified):

```
ChatCompletion(  
    id='chatcmpl-abc123',  
    model='llama-3.3-70b-versatile',  
    choices=[Choice(message=Message(role='assistant',  
    content='A HEAP is a tree-based data structure...'))],  
    usage=Usage(prompt_tokens=8, completion_tokens=142, total_tokens=150)  
)
```


Quick Glossary — Terms You'll See Everywhere

Temperature	A setting (usually 0-2) controlling randomness. Low temperature = focused, predictable answers. High temperature = more creative, varied answers.
System Prompt	A special instruction (role: 'system') set once to steer the model's personality, tone, or rules for the entire conversation.
Streaming	Receiving the response token-by-token as it's generated, instead of waiting for the full answer — this is why chatbots appear to 'type' their reply.
Hallucination	When a model confidently generates text that sounds correct but is factually wrong, because it is predicting plausible text, not looking up facts.
RAG (Retrieval-Augmented Generation)	A technique where relevant documents are fetched from a database and inserted into the prompt, so the model answers using real data instead of only its trained memory.
Agent	An LLM setup that can take actions (call tools, search the web, run code) in a loop, rather than just answering with a single block of text.
Fine-tuning	Further training an existing model on a specific, smaller dataset so it specializes in a particular task or style.
Embedding	A numerical representation (a list of numbers) of text that captures its meaning, used to compare how similar two pieces of text are.

Summary & Where to Go Next

The full loop, in one picture

1. You write a	PROMPT
2. It's broken into	TOKENS
3. Sent with a role inside a	MESSAGES LIST
4. Delivered via an authenticated	API CALL
5. The model returns a	RESPONSE
6. You extract the	content TEXT

What you can build with just this

- A command-line Q&A; bot (like the example above).
- A multi-turn chatbot — keep appending to the messages list to preserve memory.
- A RAG system — insert retrieved document text into the prompt before sending it.
- A financial chatbot — combine deterministic calculations (Pandas) with the LLM only for explaining results in natural language, reducing hallucination.

Suggested next topics

- Multi-turn conversation memory (appending assistant replies back into messages).
- Streaming responses for a real-time 'typing' effect.
- Function calling / tool use, so the model can trigger real actions.
- Prompt engineering — system prompts, few-shot examples, output formatting.

Free to use, share, and adapt. If this helped you, consider starring the repo and contributing improvements back for the next learner.